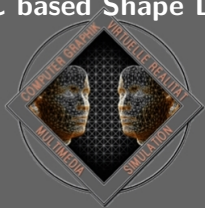


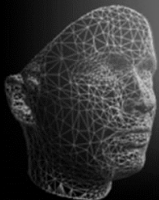
Advanced Topics in Computer Graphics II

Geometry Processing

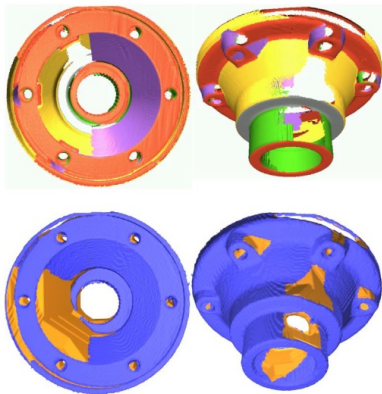
RANSAC based Shape Detection



January 26, 2026



Results:¹



Problem: Objects have natural symmetries that have to be reconstructed as well!

¹V. Lempitsky and Y. Boykov. "Global optimization for shape fitting". In: *Computer Vision and Pattern Recognition, 2007. CVPR'07. IEEE Conference on*. IEEE. 2007, pp. 1–8.



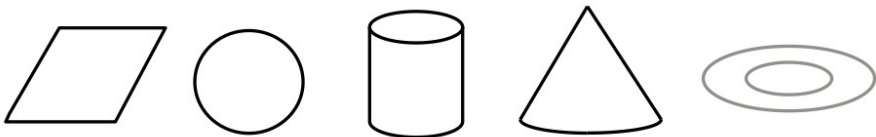
Given: (Incomplete, noisy) point cloud

$$\mathcal{P} = \{p_1, \dots, p_n\}$$

Idea: Try to detect primitives $\mathcal{S}_i$ and segment the point cloud into these primitives and the residual $\mathcal{R}$

$$\mathcal{P} = \mathcal{S}_1 \cup \dots \cup \mathcal{S}_n \cup \mathcal{R}$$

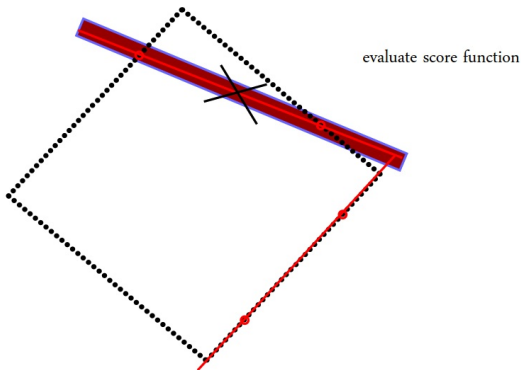
→ Not every point might be assigned to a primitive (noise)



Primitives might be: plane, sphere, cylinder, cone, torus

Problem: How to detect primitives in point clouds?

Solution: Try to find them with random sampling²



²M. A. Fischler and R. C. Bolles. "Random sample consensus: a paradigm for model fitting with applications to image analysis and automated cartography". In: *Communications of the ACM* 24.6 (1981), pp. 381–395.

Solution: Try to find them with random sampling

Now, calculate the probability that a randomly chosen sample will belong to the primitive:

- ▶ Size point cloud : N
- ▶ Size primitive : n
- ▶ Size outliers : $N - n$
- ▶ Size random sample : k

So the probability of finding the primitive is

$$\Pr(n, k) = \frac{\binom{n}{k}}{\binom{N}{k}} \stackrel{k \ll n}{\approx} \left(\frac{n}{N} \right)^k$$

The probability for finding the primitive after i candidates have been drawn equals the complementary of i consecutive failures:

$$\Pr(n, k, i) = 1 - (1 - \Pr(n, k))^i$$

The probability for finding the primitive after i iterations is

$$\Pr(n, k, i) = 1 - (1 - \Pr(n, k))^i$$

Solving for i tells us the number of candidates i required to detect shapes of size n with a probability $\Pr(n, k, i) \geq p_t$ for a given probability p_t :

$$\Pr(n, k, i) = 1 - (1 - \Pr(n, k))^i$$

$$(1 - \Pr(n, k))^i = 1 - \Pr(n, k, i)$$

$$\log((1 - \Pr(n, k))^i) = \log(1 - \Pr(n, k, i))$$

$$i \cdot \log(1 - \Pr(n, k)) = \log(1 - \Pr(n, k, i))$$

This leads to the final formula

$$i = \frac{\log(1 - \Pr(n, k, i))}{\log(1 - \Pr(n, k))}$$

We can further simplify this by using $(1 - x) \leq \exp(-x)$:

$$i \geq \frac{-\log(1 - \Pr(n, k, i))}{\Pr(n, k)}$$

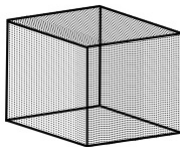
This leads to the final formula

$$i \geq \frac{-\log(1 - p_t)}{\Pr(n, k)}$$

Thus, we can compute the number of iterations i needed to find the primitive with a certain probability p_t .

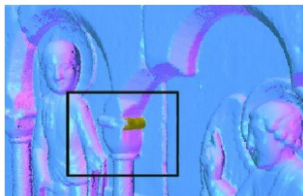
Example: If we want to find one of the planes of a regularly sampled cube with a failure rate of at most one in a million, i.e. $p_t = 1 - 10^{-6}$, the number of needed iterations is

$$i \geq \frac{-\log(1 - (1 - 10^{-6}))}{\frac{6}{216}} \approx 497$$



Problems:

- ▶ Efficient candidate generation
- ▶ Considerable computational demand without optimizations
 - ▶ Cylinder consisting of 1066 points detected among 341 587 points
 - ▶ 99% detection probability: draw 151 522 829 candidates



- ▶ The number of points belonging to the largest shape is unknown
→ When to stop sampling?
- ▶ High computational costs for checking if a randomly chosen shape fits the given data (evaluation of a score function)

Further Optimizations: Efficient candidate generation

- ▶ Probability of detecting a shape

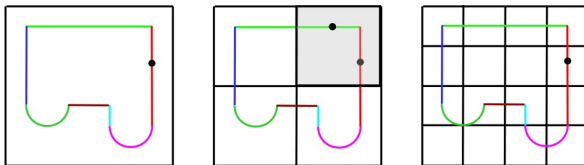
$$\Pr(n, k) = \frac{\binom{n}{k}}{\binom{N}{k}} \stackrel{k \ll n}{\approx} \left(\frac{n}{N}\right)^k$$

→ keep k small!

- ▶ Use minimal number of points to define a primitive:
 - ▶ Plane: 3 parameters
 - ▶ Sphere: 4 (3 center, 1 radius)
 - ▶ Cylinder: 4 (3 location and orientation, 1 radius)
 - ▶ Cone: 6 (3 location, 2 orientation, 1 opening angle)
 - ▶ Torus: 7 (3 location, 2 orientation, 2 radii)
- ▶ Each 3D-point on 2D surface delivers only one parameter
- ▶ Use estimated normal as additional information

Efficient candidate generation

- Use points and normals
- Use octrees to reduce number of possible candidates



- Assuming that there is a level d in the octree such that more than 50% of the points belong to a primitive Ψ of size n , the probability for finding it drawing k points $p_1, \dots, p_k$ from a local neighborhood C is

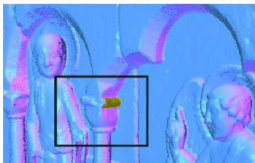
$$\Pr(n, k) = \Pr(p_1 \in \Psi) \cdot \Pr(p_2, \dots, p_k \in \Psi \mid p_1 \in \Psi \wedge p_2, \dots, p_k \in C)$$

with

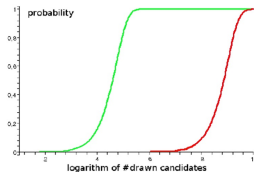
$$\Pr(p_2, \dots, p_k \in \Psi \mid p_1 \in \Psi \wedge p_2, \dots, p_k \in C) = \frac{\binom{|C|/2}{k-1}}{\binom{|C|}{k-1}} = \left(\frac{1}{2}\right)^{k-1}$$

- ▶ Stop sampling if the required number of trials to detect the largest primitive found so far is already sufficient to detect a primitive of this size with the predefined threshold probability
- ▶ Decrease cost for evaluation of score function

Example:



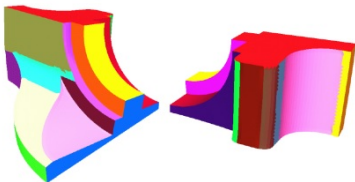
Cylinder consisting of 1066 points
detected among 341 587 points



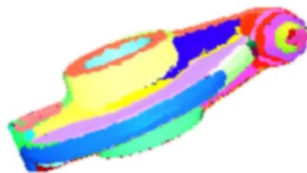
Candidates for 99% detection probability

- ▶ Without optimization: 151 522 829
- ▶ With optimization: 64 929

Results:



- 23 shapes, 0,57sec, 12 k points



- 67 shapes, 3sec., 40k points



- ~70 shapes, ~15sec., 400 kpoints

Results:

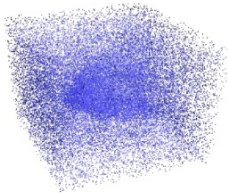
Choir screen

2Mpoints with
estimated normals,
81 shapes, 20 sec.

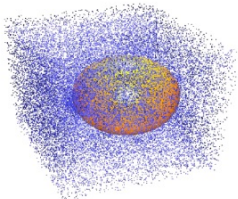


Results:

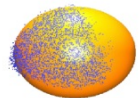
Noise



Points



Sphere

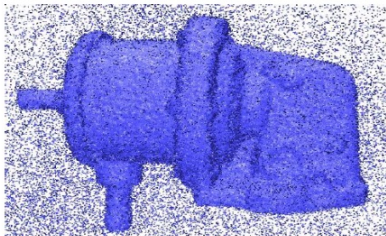


Associated points

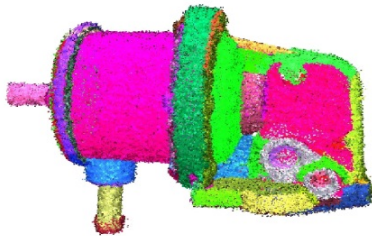
Points on an octant of a sphere distorted by synthetic gaussian noise with a σ of 10% relative to the sphere diameter and 80% outliers.

Results:

Noise



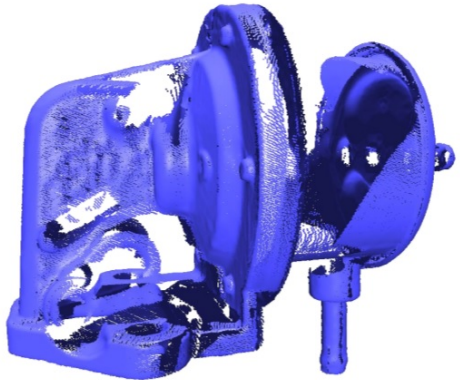
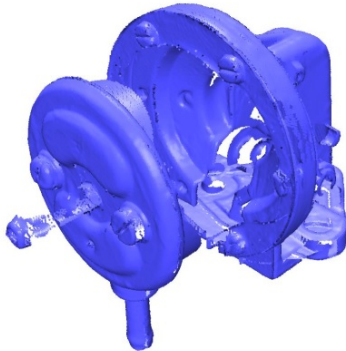
Point cloud of oil pump distorted with Gaussian noise.
In addition to the Gaussian noise 10% outliers are added.



Detection result. Shapes colored randomly.



Question: What can we reconstruct from this?





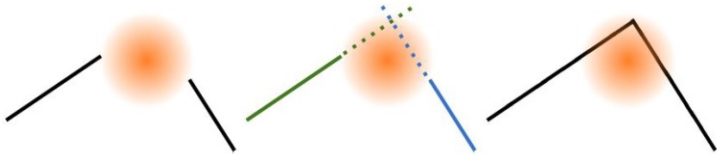
Idea: Complete the objects by fitting primitives³

Extend detected primitives such that

- ▶ Closed surface is created
- ▶ Sharp features are extended and inferred

Allow idealized reconstruction

- ▶ Recover exact primitives
- ▶ Ignore noise or surface detail

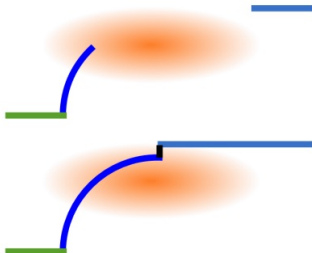


³R. Schnabel, P. Degener, et al. "Completion and Reconstruction with Primitive Shapes". In: *Computer Graphics Forum (Proc. of Eurographics)* 28.2 (2009), pp. 503–512.



Difficulties:

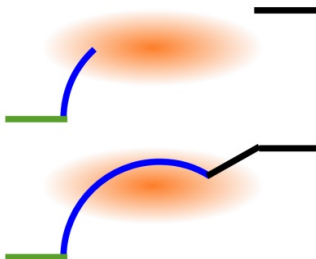
- ▶ **Inexact primitives**
- ▶ Missing primitives
- ▶ Holes with multiple boundaries
- ▶ Plausible resolution of ambiguities





Difficulties:

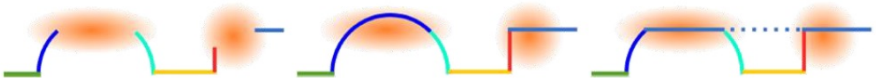
- ▶ Inexact primitives
- ▶ **Missing primitives**
- ▶ Holes with multiple boundaries
- ▶ Plausible resolution of ambiguities





Difficulties:

- ▶ Inexact primitives
- ▶ Missing primitives
- ▶ **Holes with multiple boundaries**
- ▶ Plausible resolution of ambiguities





Difficulties:

- ▶ Inexact primitives
- ▶ Missing primitives
- ▶ Holes with multiple boundaries
- ▶ **Plausible resolution of ambiguities**





Variational approach:

- ▶ Energy functional incorporating primitives' guidance
- ▶ Arbitrary number of primitives
- ▶ Arbitrarily complicated intersections
- ▶ Arbitrary number of hole boundaries
- ▶ Plausible results for inexact/missing primitives

Can create idealized reconstruction

- ▶ Recovers sharp features
- ▶ Robust to noise and outliers

Can also reconstruct detail geometry

- ▶ Handle surface parts not approximated by primitives



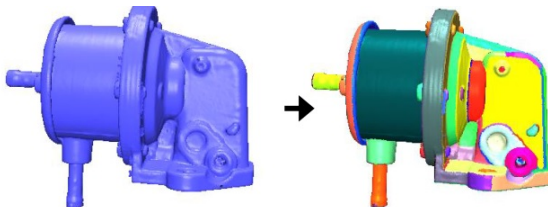
Decompose the point cloud $\mathcal{P}$ into

- ▶ Subsets associated with a primitive (support)
- ▶ Subset of remaining points

$$\mathcal{P} = \mathcal{S}_1 \cup \dots \cup \mathcal{S}_n \cup \mathcal{R}$$

RANSAC approach⁴

- ▶ Planes, spheres, cylinders, cones, tori
- ▶ Other methods possible



⁴R. Schnabel, R. Wahl, et al. "Efficient RANSAC for Point-Cloud Shape Detection". In: *Computer Graphics Forum* 26.2 (2007), pp. 214–226.



Intuition: Modify surface area

- ▶ Cheap when on primitive
- ▶ Expensive otherwise

Objective: Minimizing surface

Constraint: Primitives must be connected

Solution: Reconstruction via energy minimization

$$\begin{aligned} E(S) &= \underbrace{E_a(S)}_{\text{surface area}} - \underbrace{E_p(S)}_{\text{primitive adherence}} + \underbrace{E_c(S)}_{\text{inside/outside constraints}} \\ &= \int_S dA - \int_S H(\langle \mathbf{n} | \mathbf{v} \rangle) dA + E_c(S) \end{aligned}$$

where

$$H(x) = \begin{cases} 1 & \text{if } x > 0 \\ 0 & \text{o/w} \end{cases}$$

and $\mathbf{n}$ be the normals of S and $\mathbf{v}$ be a vector field defined for S



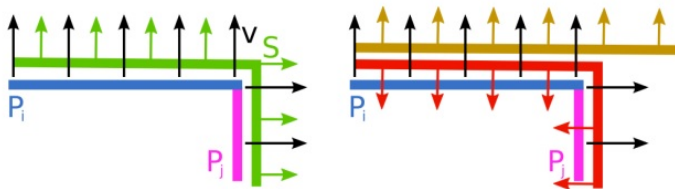
Solution: Reconstruction via energy minimization

$$\begin{aligned} E(S) &= \underbrace{E_a(S)}_{\text{surface area}} - \underbrace{E_p(S)}_{\text{primitive adherence}} + \underbrace{E_c(S)}_{\text{inside/outside constraints}} \\ &= \int_S dA - \int_S H(\langle \mathbf{n} | \mathbf{v} \rangle) dA + E_c(S) \end{aligned}$$

where

$$H(x) = \begin{cases} 1 & \text{if } x > 0 \\ 0 & \text{o/w} \end{cases}$$

and $\mathbf{n}$ be the normals of S and $\mathbf{v}$ be a vector field defined for S

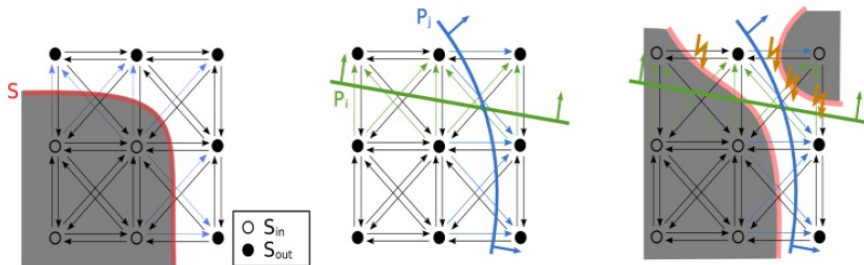




Problem: Energy has many local minima

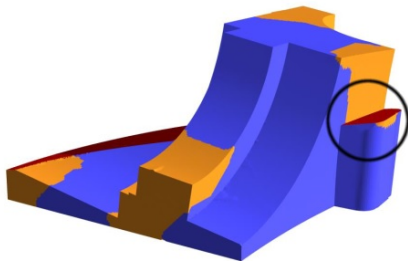
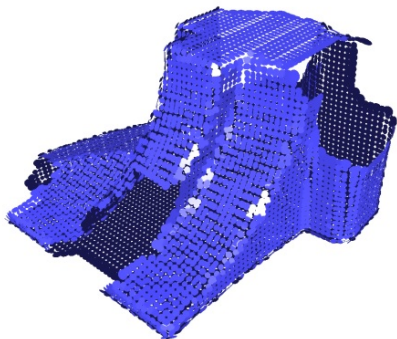
Idea: Find discrete globally optimal solution with graph-cut

- For now: ignore connectivity
- Define discrete edge cost on volumetric grid





Results: Without connectivity

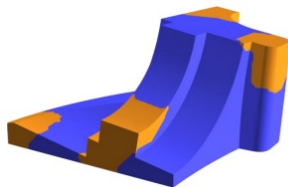
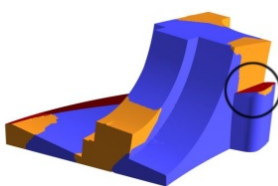
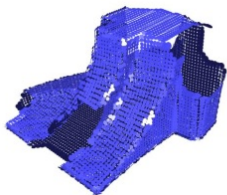




Each cut-edge is representative for local surface patch. Cut-edges intersected by primitive must be connected.

Iterate:

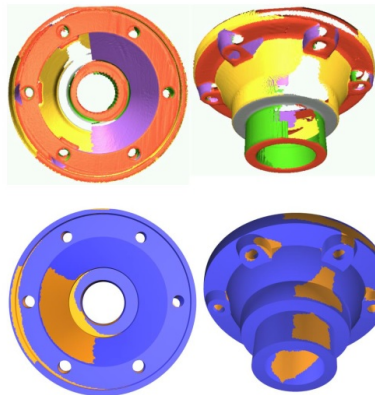
- ▶ Compute reconstruction with graph-cut
- ▶ Detect violations of connectivity constraint with graph traversal
- ▶ Increase cost of violating edges





Completion with primitives is now possible when reconstruction adheres to primitives everywhere.

Question: How to handle detail?





Detail reconstruction: Based on an approach by Lempitsky and Boykov (2007)

- Derive vector field $\vec{V}$ from input points

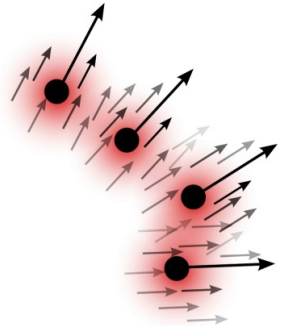
New energy term:

$$E(S) = \underbrace{E_a(S)}_{\text{surface area}} - \underbrace{E_p(S)}_{\text{primitive adherence}} + \underbrace{\lambda E_u(S)}_{\text{vector field}}$$

with

$$E_u(S) = \int_S \langle \mathbf{n} | \mathbf{v} \rangle dA = \int_{\text{interior}(S)} \text{div } \mathbf{v} dV$$

- Minimum can also be found with graph-cut
- Still fills holes with primitives



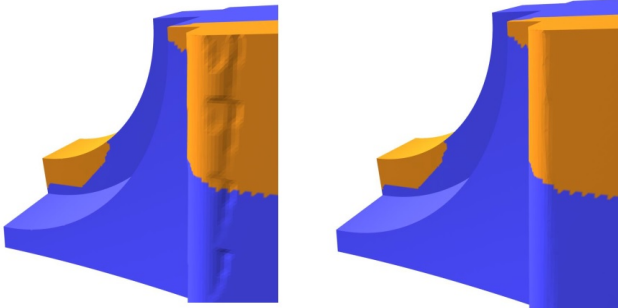


Extract the isosurface using Extended Marching Cubes.

Close to primitive transitions:

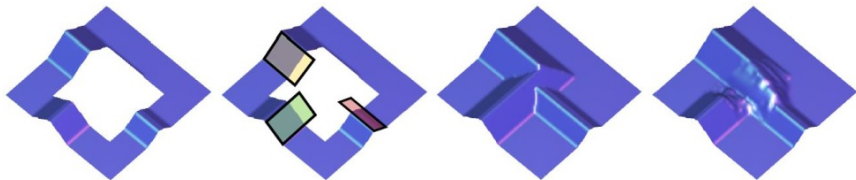
- ▶ Cut-edges intersected by multiple primitives
- ▶ Which normal/intersection to pick?

Results:

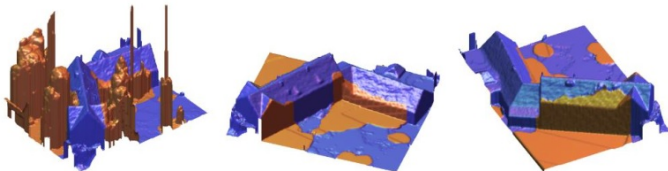




Results:

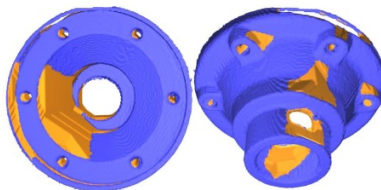
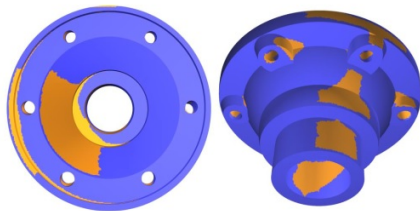
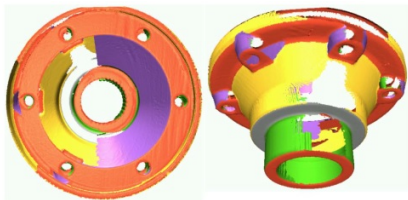


[JIA ET AL. 2007]





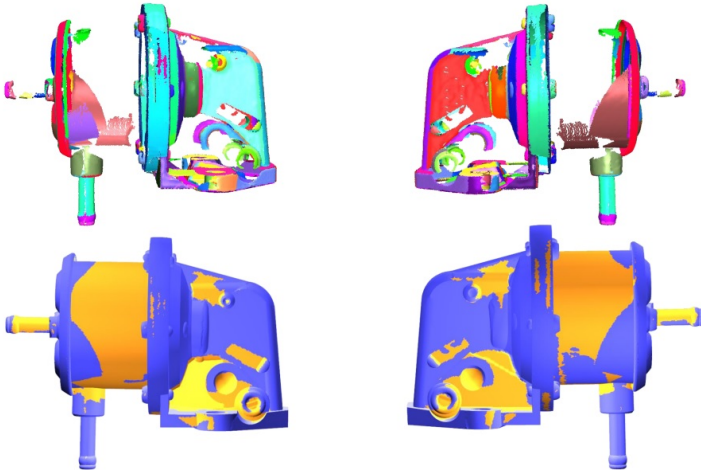
Results:



[LEMPITSKY AND BOYKOV 2007]

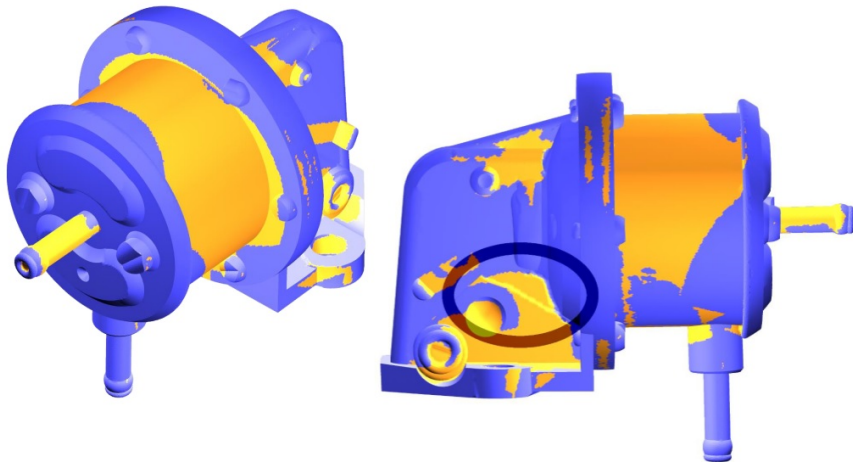


Results:





Results:





Conclusions:

- ▶ Automatically completes holes by extending primitives
- ▶ Resolves ambiguities
- ▶ Reconstructs and extends sharp features
- ▶ Infers sharp features in holes from primitive intersections
- ▶ Can generate idealized or detailed model



Has Deep Learning Replaced RANSAC?

Classical RANSAC (1981–2016)

- ▶ Hand-crafted features
 - ▶ Point Pair Features (PPF)
 - ▶ FPFH, SHOT
 - ▶ Geometric descriptors
- ▶ Random sampling
 - ▶ Uniform random
 - ▶ Octree-based
 - ▶ Edge-based
- ▶ Geometric verification
 - ▶ Distance thresholds
 - ▶ Inlier counting

Strength: Works on any geometry,
no training data needed

Modern Hybrid Methods (2017–2025)

- ▶ **Learned features**
 - ▶ PointNet++
 - ▶ FCGF (Fully Conv. Geom.)
 - ▶ SpinNet, DCP
- ▶ **Learned sampling**
 - ▶ Saliency-guided
 - ▶ Keypoint detection
 - ▶ Confidence-based
- ▶ **RANSAC verification**
 - ▶ Still needed!
 - ▶ Geometric consistency
 - ▶ Robust estimation

Strength: Better features,
but RANSAC framework remains!

Key Insight: The RANSAC *framework* is fundamental — only the *features* have changed!

Modern methods use learned components *within* the RANSAC paradigm.

Beyond Classical Optimizations: New acceleration strategies from the last decade

1. GPU Acceleration (2018+)

- ▶ Parallel hypothesis generation and testing
- ▶ CUDA-based implementations (CURANSAC, GPU-RANSAC)
- ▶ Speedup: **100–1000×** vs. naive CPU implementation
- ▶ Real-time performance: <100ms for typical point clouds

2. Learned Sampling (2019+)

- ▶ Neural networks predict point *saliency* / *importance scores*
- ▶ RANSAC samples from high-confidence regions
- ▶ Examples:
 - ▶ BoundED statistics (Bode et al. 2023) — statistical descriptors for edge detection
 - ▶ 3DFeat-Net (Yew & Lee 2018) — learned keypoint detection
 - ▶ D3Feat (Bai et al. 2020) — dense feature extraction
- ▶ Result: Fewer iterations needed, faster convergence

3. Progressive & Adaptive Methods (2020+)

- ▶ **PROSAC**: Sample from sorted quality scores (non-uniform sampling)
- ▶ **NAPSAC**: Sample from spatial neighborhoods
- ▶ **Adaptive termination**: Stop when confidence threshold reached
- ▶ **Multi-hypothesis tracking**: Maintain multiple candidate solutions

Take-home: RANSAC continues to evolve — it's not a static 1981 algorithm!
Modern variants combine classical robustness with learned efficiency.

Industry Applications

► Robotics

- Amazon warehouse: Bin picking
- Boston Dynamics: Object manipulation
- Mobile robots: Localization

► Autonomous Vehicles

- Waymo: Point cloud localization
- Tesla: Vision-based odometry
- Sensor calibration pipelines

► Industrial Inspection

- Boeing: Automated quality control
- Siemens: CAD-to-scan matching
- Precision metrology

► Medical Imaging

- Surgical navigation systems
- CT/MRI registration
- Implant placement planning

Academic Benchmarks

Still use RANSAC as baseline:

► 3DMatch / 3DLoMatch

- Indoor scene registration
- RANSAC + FPFH baseline
- All learning methods compared against it

► KITTI Odometry

- Autonomous driving dataset
- Geometric methods competitive
- RANSAC-based in top-20

► ETH/TUM RGB-D

- Dense reconstruction
- RANSAC for loop closure

Why? RANSAC provides:

- ✓ Mathematical guarantees
- ✓ Zero-shot generalization
- ✓ Interpretable failures

Recent Papers Using RANSAC: SC²-PCR (CVPR'24), RoReg (ICCV'23), PointMBF (CVPR'23), RegTR (CVPR'22), PointDSC (CVPR'21)

Citation Statistics: Fischler & Bolles (1981) still receives ~1,500 citations/year!

If RANSAC were obsolete, citations would be declining. Instead, they remain strong.



Beyond Local Search: Global Template Matching





Methods like ICP and CPD are **local solvers**—they require a good initial pose to reach the global minimum.

The Global Challenge

How do we find a template (object) in a large, cluttered, and partially occluded scene without *any* initial guess?

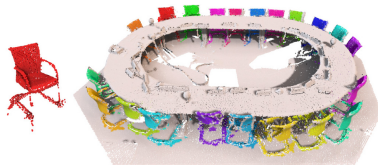


Fig. 4. Detected instances of chairs around a table. Individual instances are illustrated with different colors. In contrast to the scene the template was scanned completely and is shown on the left.

Global Template Matching: Vock et al. (2019)⁵

- ▶ **Approach:** A RANSAC-based framework using **Point Pair Features (PPF)**.
- ▶ **Core Innovation:** Intelligent "Salient" sampling and an "Early Exit" scoring strategy.
- ▶ **Result:** A robust global pose that serves as the perfect initialization for CPD or ICP.

⁵R. Vock et al. "Fast template matching and pose estimation in 3D point clouds". In: *Computers & Graphics* 79 (2019), pp. 36–45.

The Pipeline: From Hashing to Validation

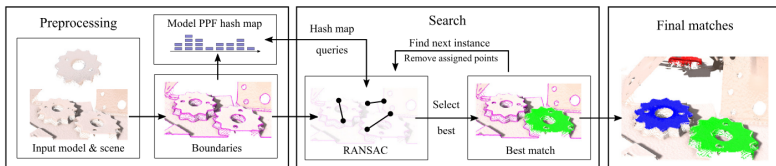


Fig. 1. Overview of our computation pipeline. Boundary/edge points are detected (left) and point pairs are sampled in order to generate transformation hypotheses in a RANSAC loop (middle). The best candidates with respect to a match fitness score then yield the final transformation set (right).

The algorithm follows a "Hypothesize-and-Verify" loop:

1. **Preprocessing:** Identify **edge points** and store template geometry in a **Geometric Hash Map**.
2. **Sampling:** Select a pair of points in the scene.
3. **Voting/Matching:** Query the hash map to find similar pairs on the template.
4. **Hypothesis:** Generate a 4x4 rigid transformation T for each match.
5. **Validation:** Score the transformation using a **Voxel Distance Field**.

Two performance bottlenecks: $O(N^2)$ point pair combinations and expensive scoring of every hypothesis.

Defining the Point Pair Feature (PPF)

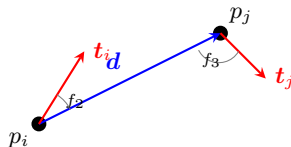
A point pair (p_i, p_j) is converted into a 4D descriptor f that describes their **relative geometry**. This descriptor is **rigid-invariant** (it doesn't change if the object moves).

The 4D Feature Vector:

$$f(p_i, p_j) = \begin{pmatrix} \|d\|_2 \\ \angle(t_i, d) \\ \angle(t_j, d) \\ \kappa_{min}/\kappa_{max} \end{pmatrix}$$

where:

- ▶ $d = p_j - p_i$ (displacement)
- ▶ t_i, t_j are the **edge directions**
- ▶ f_4 is the **curvature ratio** at p_i



Geometrically, we describe the distance and how the edges "point" toward each other.

Why this is powerful

Every pair in the template with similar (f_1, f_2, f_3, f_4) is stored in the same **Hash Bin**. A single scene query retrieves *all* potential matching locations on the template in $O(1)$ time.



When we query the Hash Map with a scene pair f_{scene} , we rarely get a single match.

Definition: Branching Factor (B)

The number of template pairs $\{p_k, p_l\}$ stored in the same discrete hash bin.

$$B = \text{count}(\text{Hash}(f_{scene}) \text{ in Template Map})$$

The "Search Explosion":

- ▶ If we take S samples in the scene, the total number of hypotheses to verify is:

$$\text{Total Hypotheses} = S \times B$$

- ▶ **Geometric Ambiguity:** If the feature is not unique (e.g., matching points on a flat plane), B can be in the thousands.
- ▶ **Impact:** This is why scoring takes 95% of the runtime. We are drowning in "mathematically valid" but "geometrically wrong" guesses.



Instead of sampling all N points, we restrict ourselves to **Edge Points** $\mathcal{E} \subset P$.

Feature Extraction

We compute a 5D local descriptor $f(p)$ using the **Weighted Covariance Tensor** (the same tools used for PCA):

$$f(p) = [\text{angle gap, centroid dist, non-planarity, sphericity, variation}]^T$$

- ▶ **Classification:** We cluster points into "Edge" vs "Non-Edge" using k -means.
- ▶ **Why it works:** Edges are more descriptive and stable under occlusion than flat surface points.
- ▶ **Complexity Reduction:** Reducing the sample pool from N to N_{edge} drastically minimizes the RANSAC search space.



Standard scoring requires a Nearest Neighbor (NN) search for every point in the template.

The Vock et al. approach:

- ▶ Precompute a **Voxelized Distance Field** for the template.
- ▶ Each voxel stores the index of the closest template point.
- ▶ **Result:** NN-lookup becomes a **constant time** $O(1)$ array access.

The Alignment Score

$$\sigma_T(Q) = \sum_{p \in Q} \exp\left(-\frac{\|Tp - \text{NN}(Tp)\|^2}{2k\rho}\right)$$

Where ρ is the resolution and k is a tolerance factor.



Most RANSAC hypotheses are "garbage." Fully scoring them is a waste of time.

Hypergeometric Early Exit

We score a small random subset (e.g., 5%) of the points first. Using the **Hypergeometric Distribution**, we estimate the **upper bound** of the possible final score.

- ▶ **The Test:** If the predicted upper bound is lower than our current "best" score, we terminate the scoring for this hypothesis immediately.
- ▶ **Performance:** This "Early Exit" strategy avoids full validation for over **80%** of candidate transformations.
- ▶ **Impact:** Achieves speedups of up to two orders of magnitude ($100\times$) compared to unoptimized RANSAC.



Fig. 4. Detected instances of chairs around a table. Individual instances are illustrated with different colors. In contrast to the scene the template was scanned completely and is shown on the left.

Figure: Learned correspondences are much cleaner than geometric ones.



While RANSAC-based methods (Vock et al. 2019) are robust, they rely on **hand-crafted features** (angles, distances) that can fail in extreme noise or repetitive geometry.

Modern Evolution

Instead of defining what a "good match" looks like, we **train a neural network** to learn descriptors that are invariant to rotation, noise, and density.

Three Main Categories of Learning Approaches:

1. **Deep Feature Matching:** Learning local descriptors (e.g., FCGF⁶, SpinNet⁷).
2. **End-to-End Registration:** Direct pose prediction (e.g., DCP⁸, Deep Global Registration⁹).
3. **Neural Voting:** Predicting keypoints and object centers (e.g., PVN3D¹⁰, FFB6D¹¹).

⁶choy2019fully.

⁷ao2021spinnet.

⁸Wang and Solomon, 2019.

⁹Choy et al., 2020.

¹⁰He, W. Sun, et al., 2020.

¹¹He, H. Sun, et al., 2021.



- **Concept:** Use a 3D Convolutional Neural Network (CNN) or a Transformer to map every point p to a high-dimensional feature vector $f_p \in \mathbb{R}^{32}$.
- **Why it beats RANSAC:** Hand-crafted PPFs only see two points. Deep features see the **semantic context** (e.g., "this point is part of a chair leg").

GeoTransformer (2022)^a:

- Uses **Self-Attention** to learn the relative geometry of the whole cloud.
- Achieves an inlier ratio of $> 90\%$.
- RANSAC becomes trivial or unnecessary.

^aQin et al., 2023.

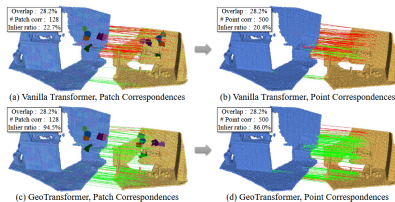


Figure: Learned correspondences are much cleaner than geometric ones.



For template matching, modern methods often use **Deep Hough Voting**.

- ▶ **The Method:** Every point in the scene predicts a 3D offset vector pointing toward the **object center**.
- ▶ **Comparison to Vock et al.¹²:**
 - ▶ Vock et al. use edge points to propose transformations.
 - ▶ Neural methods use **every point** to "vote" for the pose.
- ▶ **Robustness:** Even if 70% of the object is occluded, the remaining 30% of points still know where the center is.

Key Result

Methods like **PVN3D**¹³ utilize both geometry (PointNet++)¹⁴ and color (RGB) to distinguish between identical-looking geometric parts (e.g., two different types of fuses).

¹²Vock et al., 2019.

¹³He, W. Sun, et al., 2020.

¹⁴Qi et al., 2017.

Summary: Heuristics vs. Learning

Feature	RANSAC/PPF (Vock et al.)	Learning-Based
Generalization	Works on any new geometry	Needs training on similar data
Speed	Fast (if early-exit is used)	Very fast (GPU inference)
Robustness	Fails in high noise/symmetry	Extremely robust to noise
Semantics	Only sees geometry	Understands "Object Types"
Hardware	CPU-friendly	Requires high-end GPU

The Verdict

Heuristic methods remain the gold standard for industrial metrology and "out-of-the-box" tasks where no training data exists. **Learning methods** dominate in robotics and real-time AR where speed and semantic understanding are critical.



Classical Point Pair Features (PPF) are "semantically blind"—they cannot distinguish between an edge on a gear and an edge on a table if the local angles match.

The Idea: Augmented Descriptors

Combine the **Geometric Rigor** of Vock et al. with **Deep Semantic Context**. Instead of a 4D PPF, we use a Hybrid Descriptor:

$$f_{\text{hybrid}}(p_i, p_j) = \left[\underbrace{\|d\|, \angle_1, \angle_2, \angle_3}_{\text{Geometric (PPF)}}, \underbrace{\Phi(p_i), \Phi(p_j)}_{\text{Learned Context}} \right]$$

Concentrating on "Promising" Pairs:

- ▶ Use a lightweight backbone (e.g., PointNet++ or KPConv) to compute a per-point feature $\Phi(p)$.
- ▶ **Filtering:** Only propose transformation hypotheses if the learned features $\Phi(p_{\text{scene}})$ and $\Phi(p_{\text{template}})$ are highly similar.



The goal of adding **BoundedED statistics** is to make the descriptor so unique that the Branching Factor B collapses toward 1.

Vock et al. (4D PPF):

- ▶ "Find all pairs with distance d and angles α, β ."
- ▶ **Result:** High B (many accidental matches).

Hybrid (14D Augmented):

- ▶ "...and both points must be on a **sharp convex ridge**."
- ▶ **Result:** Low B (only true edges match).

Statistical Pruning

By increasing the dimensionality from 4D to 14D, we perform **Semantic Filtering** during the hash-lookup. This removes the "garbage" hypotheses *before* the expensive scoring phase begins.

Mathematical Result: The complexity of the RANSAC loop is reduced from $O(S \cdot B_{\text{classical}})$ to $O(S \cdot B_{\text{hybrid}})$, where $B_{\text{hybrid}} \ll B_{\text{classical}}$.



In the original Vock et al. approach, a geometric query might return M potential matches in the hash map, leading to a massive search space.

Classical PPF matching:

- ▶ Query returns **all** pairs with similar distances/angles.
- ▶ High number of "false positive" hypotheses.
- ▶ RANSAC requires more iterations to find the true inliers.

Learned Hybrid matching:

- ▶ Query is filtered by semantic similarity.
- ▶ Only "semantically consistent" pairs are used.
- ▶ **Result:** Drastic reduction in the number of RANSAC iterations.

Concentrated Sampling

Instead of random sampling, we sample points based on a **Learned Saliency Score**. We prioritize points that the network identifies as "uniquely descriptive" for that specific object.

Note that this hybrid approach is similar to *Point-Pair Feature Networks*¹⁵ where they specifically use a *PointNet-style* architecture to encode the PPFs.

¹⁵Deng et al., 2018.



Summary: Why this Hybrid Approach Wins



- ▶ **Efficiency:** Learned features act as a "gatekeeper," ensuring that RANSAC only spends time on the most promising candidates.
- ▶ **Robustness:** We keep the **Voxel-based Scoring** and **Early Exit** from Vock et al., providing a mathematical guarantee of fit that pure black-box deep learning often lacks.
- ▶ **Interpretation:** Unlike end-to-end networks, we can still "see" which point pairs created the match, making the system easier to debug in industrial settings.

Takeaway for the Lecture

Modern Registration is moving toward **Neural-Augmented Geometry**: Using Deep Learning to find *where* to look, and Classical Geometry to calculate *exactly* how to align.



Instead of hand-crafted angles, we can utilize **Local Neighborhood Statistics**¹⁶ to enrich our point descriptors.

The 10D Statistical Descriptor

For each point, we split the neighborhood by the tangent plane and compute:

- ▶ **Sub-set Covariances** ($\sigma_{up}, \sigma_{low}$): Captures local "sharpness."
- ▶ **Relative Means** (d_i): Detects geometric steps or ridges.
- ▶ **Point-to-Mass Distance** (s_i): Identifies boundaries/holes.

Simultaneous Computation:

- ▶ The same 10D vector $\hat{x}_i$ is used to **classify** the point (Edge/Boundary) and **describe** it for matching.
- ▶ No extra computational cost for the descriptor if edge detection is already performed.

¹⁶L. Bode et al. "Bounded: Neural Boundary and Edge Detection in 3D Point Clouds via Local Neighborhood Statistics". In: *ISPRS Journal of Photogrammetry and Remote Sensing* 205 (2023), pp. 334–351.



Problem: Standard PPF matches points based on distance only, leading to many "Geometric False Positives" in cluttered scenes.

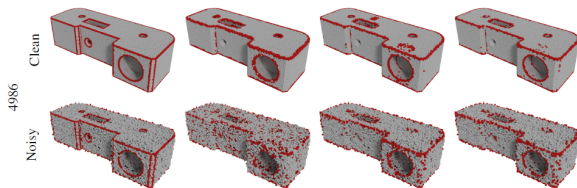


Figure: BoundED (right two columns) stays clean in noisy regions where others fail.

- ▶ **Class Consistency:** Only match points of the same type (e.g., *Sharp Edge* $\rightarrow$ *Sharp Edge*).
- ▶ **Directional Alignment:** Use the *Edge Direction* (e_2) to prune transformations early.
- ▶ **Noise Filtering:** Statistical robustness prevents matching on sensor artifacts (Slide 15).

Conclusion

By combining **Vock et al.**'s RANSAC framework with **BoundED**'s statistical descriptors, we should achieve a global registration that is semantically aware and highly robust to sensor noise.



- [1] L. Bode, M. Weinmann, and R. Klein. “BoundED: Neural Boundary and Edge Detection in 3D Point Clouds via Local Neighborhood Statistics”. In: *ISPRS Journal of Photogrammetry and Remote Sensing* 205 (2023), pp. 334–351.
- [2] C. Choy, W. Dong, and V. Koltun. “Deep Global Registration”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2020, pp. 2514–2523.
- [3] H. Deng, T. Birdal, and S. Ilic. “PPFNet: Global Context Aware Local Features for Robust 3D Point Matching”. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2018, pp. 195–205.
- [4] M. A. Fischler and R. C. Bolles. “Random sample consensus: a paradigm for model fitting with applications to image analysis and automated cartography”. In: *Communications of the ACM* 24.6 (1981), pp. 381–395.
- [5] Y. He, H. Sun, W. Huang, J. Chen, H. Fan, and J. Sun. “FFB6D: A Full Flow Bidirectional Fusion Network for 6D Pose Estimation”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2021, pp. 3003–3013.



- [6] Y. He, W. Sun, H. Huang, J. Liu, H. Fan, and J. Sun. “Pvn3d: A deep point-wise 3d keypoints voting network for 6dof pose estimation”. In: *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 2020, pp. 11632–11641.
- [7] V. Lempitsky and Y. Boykov. “Global optimization for shape fitting”. In: *Computer Vision and Pattern Recognition, 2007. CVPR'07. IEEE Conference on*. IEEE. 2007, pp. 1–8.
- [8] C. R. Qi, L. Yi, H. Su, and L. J. Guibas. “Pointnet++: Deep hierarchical feature learning on point sets in a metric space”. In: *Advances in neural information processing systems* 30 (2017).
- [9] Z. Qin, H. Yu, C. Wang, Y. Guo, Y. Peng, S. Ilic, D. Hu, and K. Xu. “Geotransformer: Fast and robust point cloud registration with geometric transformer”. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 45.8 (2023), pp. 9806–9821.
- [10] R. Schnabel, P. Degener, and R. Klein. “Completion and Reconstruction with Primitive Shapes”. In: *Computer Graphics Forum (Proc. of Eurographics)* 28.2 (2009), pp. 503–512.



- [11] R. Schnabel, R. Wahl, and R. Klein. “Efficient RANSAC for Point-Cloud Shape Detection”. In: *Computer Graphics Forum* 26.2 (2007), pp. 214–226.
- [12] R. Vock, A. Dieckmann, S. Ochmann, and R. Klein. “Fast template matching and pose estimation in 3D point clouds”. In: *Computers & Graphics* 79 (2019), pp. 36–45.
- [13] Y. Wang and J. M. Solomon. “Deep Closest Point: Learning Representations for Point Cloud Registration”. In: *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. 2019, pp. 3523–3532.